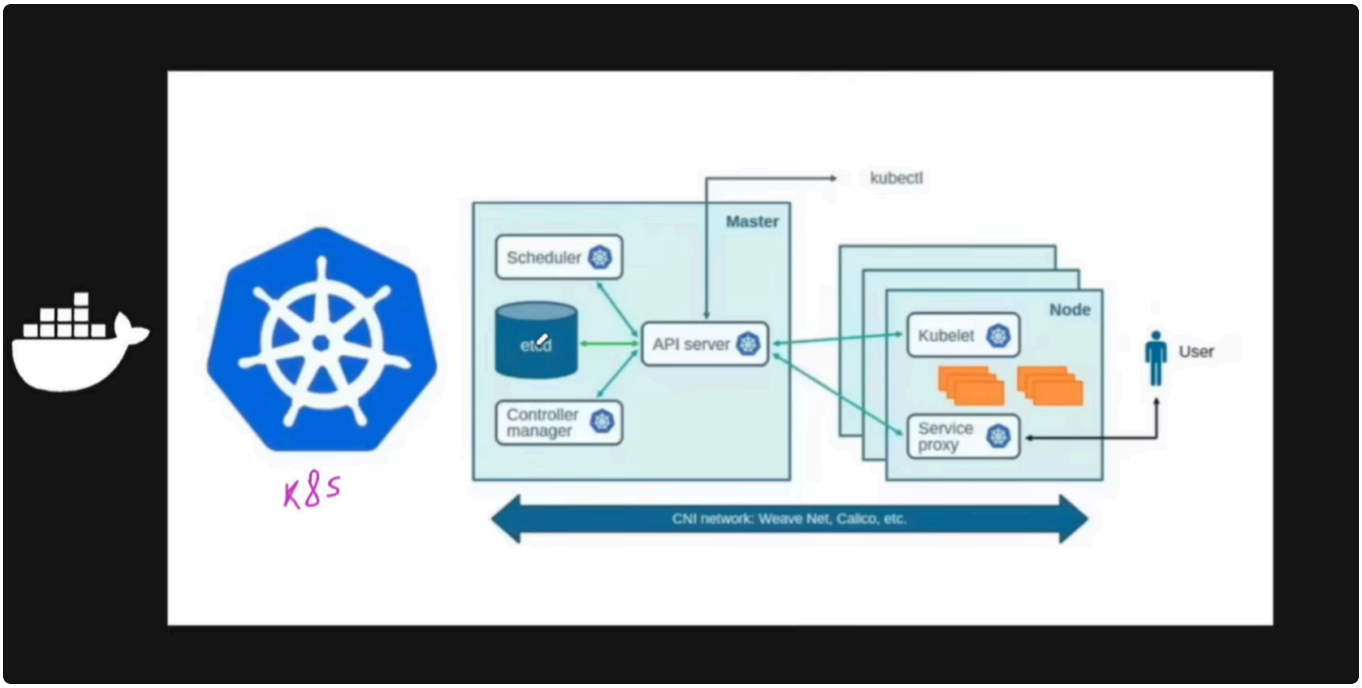
k8s in one short

core concept:

- monolithic vs microservices
- kubernetes arthitecture
- setup on local/AWS
- kubectl
- pods
- Namespaces
- Labels
- selectors
- Annotation



Kubernetes Cluster:

- Kubeadm
- MiniKube (local/EC2)
- KIND Cluster
- EKS/ AKS/ GKE

Install KIND Cluster
ref: https://github.com/LondheShubham153/kubestarter

```
ashish@Ashish:~/k8s$ kubectl version
Client Version: v1.33.3
Kustomize Version: v5.6.0
The connection to the server localhost:8080 was refused - did you specify the right host or port?
ashish@Ashish:~/k8s$ kind --version
kind version 0.33.0
ashish@Ashish:~/k8s$ docker --version
Docker version 29.7.1, build e9452d6
ashish@Ashish:~/k8s$
```

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4

nodes:
- role: control-plane
  image: kindest/node:v1.31.2
- role: worker
  image: kindest/node:v1.31.2
- role: control-plane
  image: kindest/node:v1.31.2
- role: control-plane
  image: kindest/node:v1.31.2
- role: control-plane
  image: kindest/node:v1.31.2
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    protocol: TCP
```

```
kind create cluster --name=k8s-cluster  --config=config.yml
```

```
ashish@Ashish:~/k8s/Kind_Cluster$
ashish@Ashish:~/k8s/Kind_Cluster$ kind create cluster --name=k8s-cluster --config=config.yml
Creating cluster "k8s-cluster" ...
 ✓ Ensuring node image (kindest/node:v1.31.2) 🖼
 ✓ Preparing nodes 📦 📦 📦 📦 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
 ✓ Joining worker nodes 🚜
Set kubectl context to "kind-k8s-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-k8s-cluster

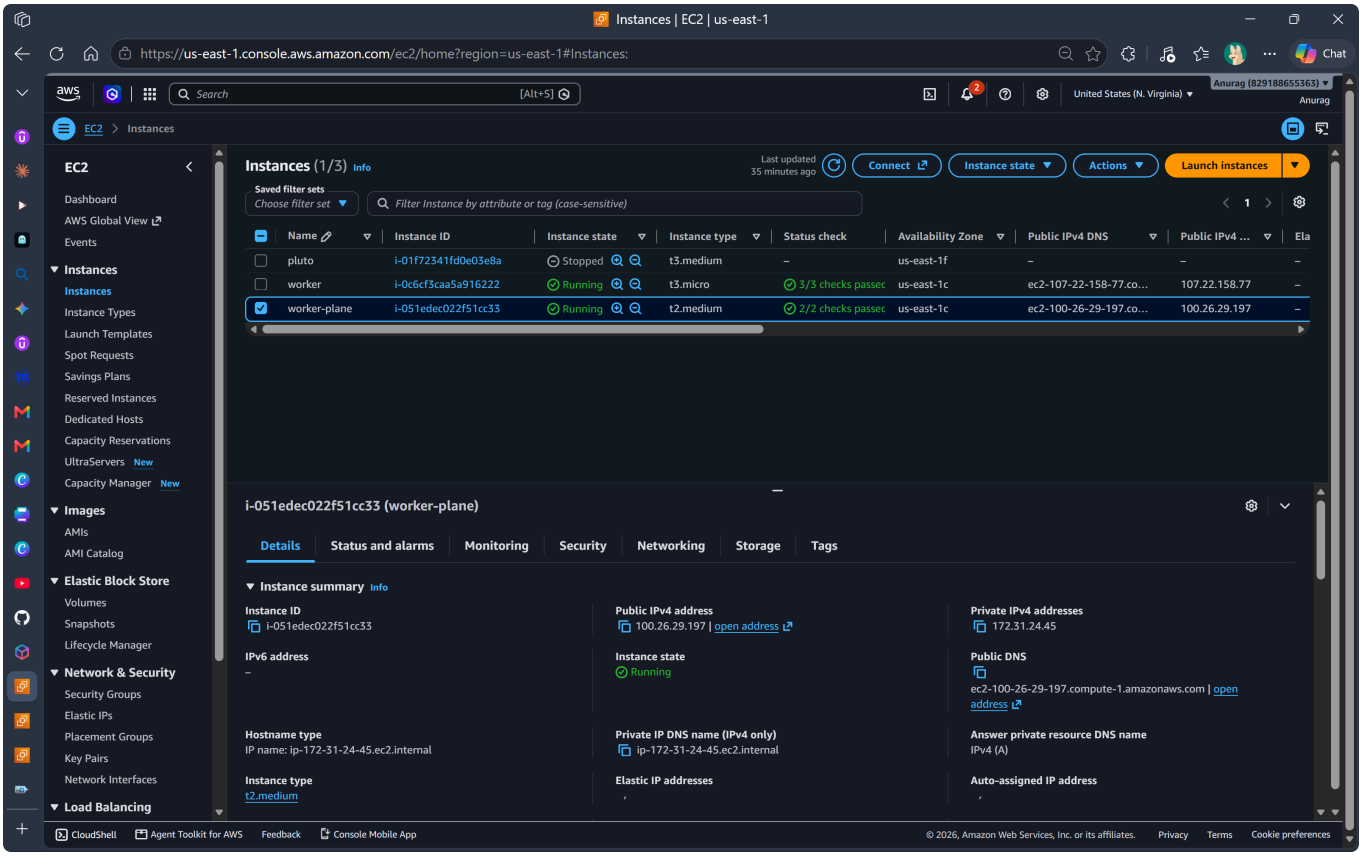
Have a question, bug, or feature request? Let us know! https://kind.sigs.k8s.io/#community
😊
```

```
kubectl get nodes
```

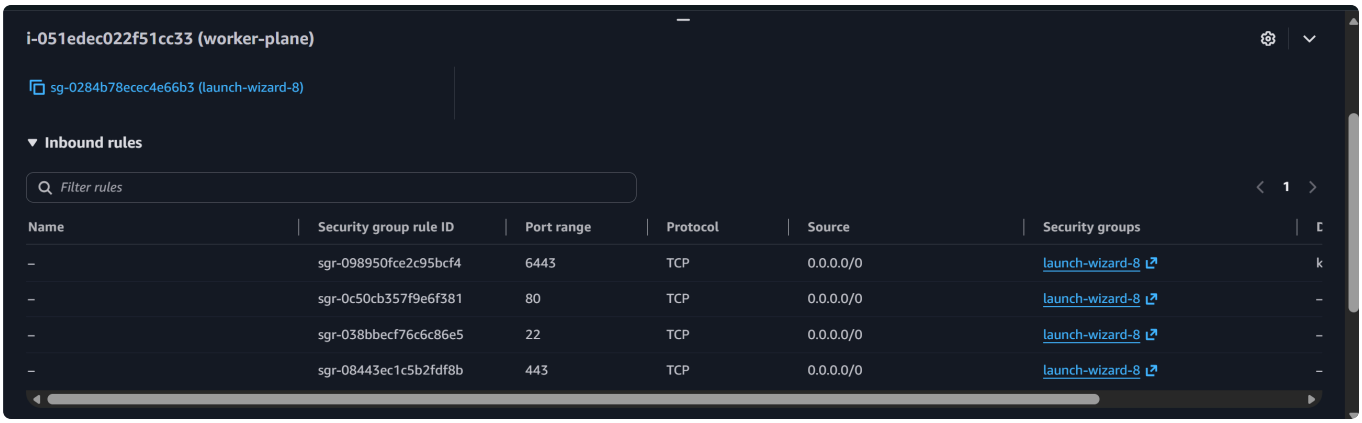
```
ashish@Ashish:~/k8s/Kind_Cluster$ kubectl get nodes
NAME                          STATUS   ROLES           AGE     VERSION
k8s-cluster-control-plane     Ready    control-plane   3m32s   v1.31.2
k8s-cluster-worker            Ready    <none>          3m17s   v1.31.2
k8s-cluster-worker2           Ready    <none>          3m16s   v1.31.2
k8s-cluster-worker3           Ready    <none>          3m17s   v1.31.2
k8s-cluster-worker4           Ready    <none>          3m16s   v1.31.2
ashish@Ashish:~/k8s/Kind_Cluster$
```

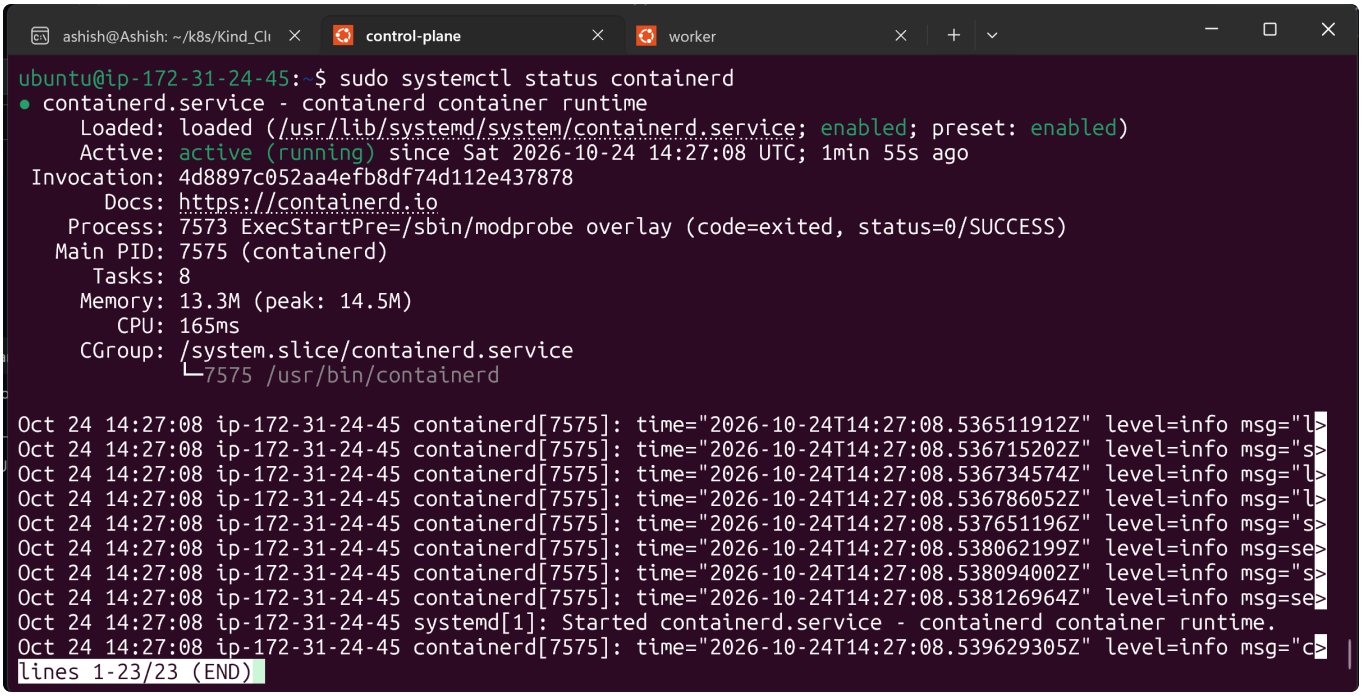
Setup kubeadm on EC2:

ref: [kubestarter/Kubeadm_Installation_Scripts_and_Documentation at main · LondheShubham153/kubestarter · GitHub]

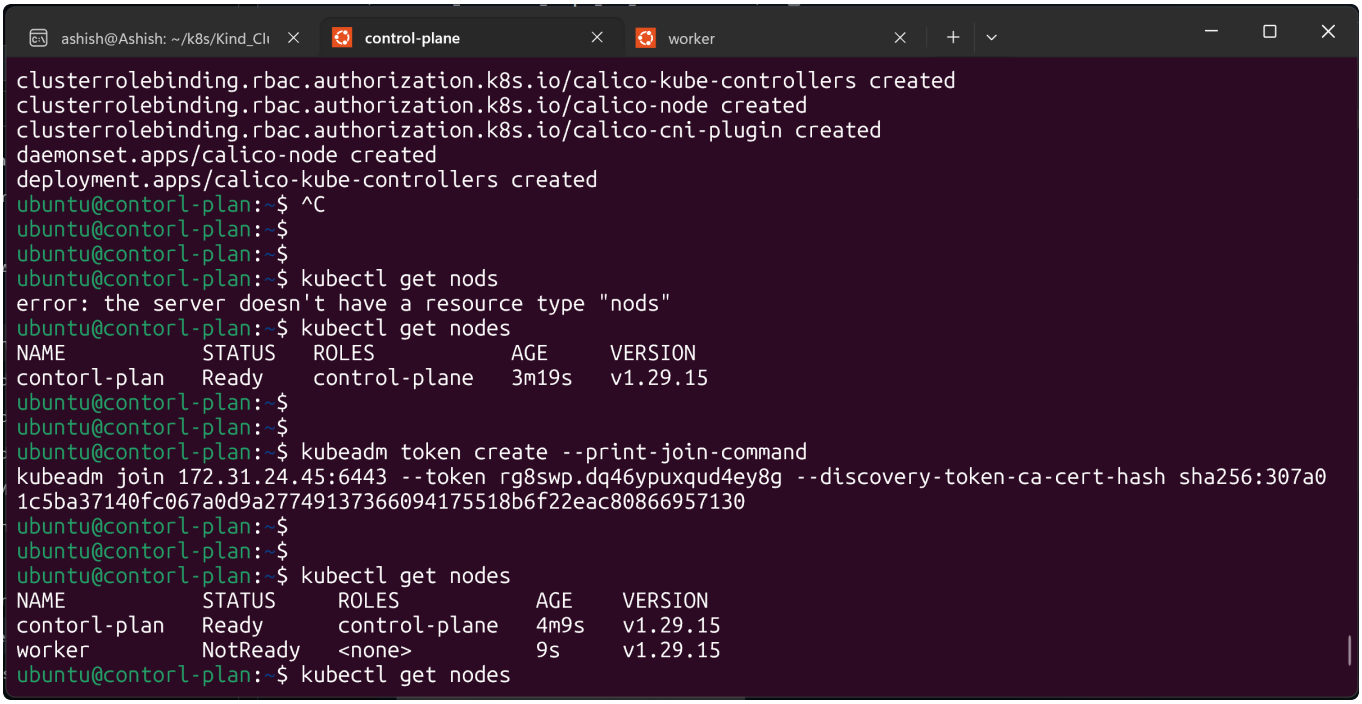


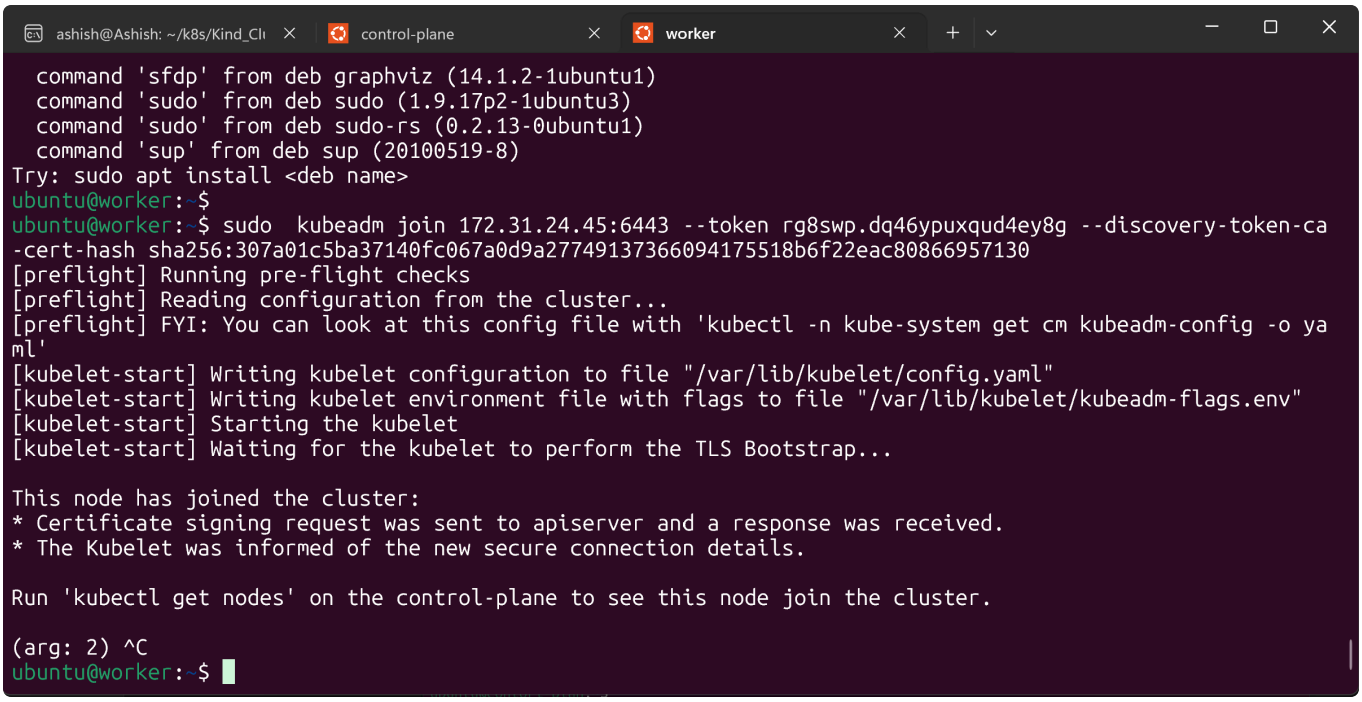
Add 6443 port as inbound for connect both node each other via a Kubeadm



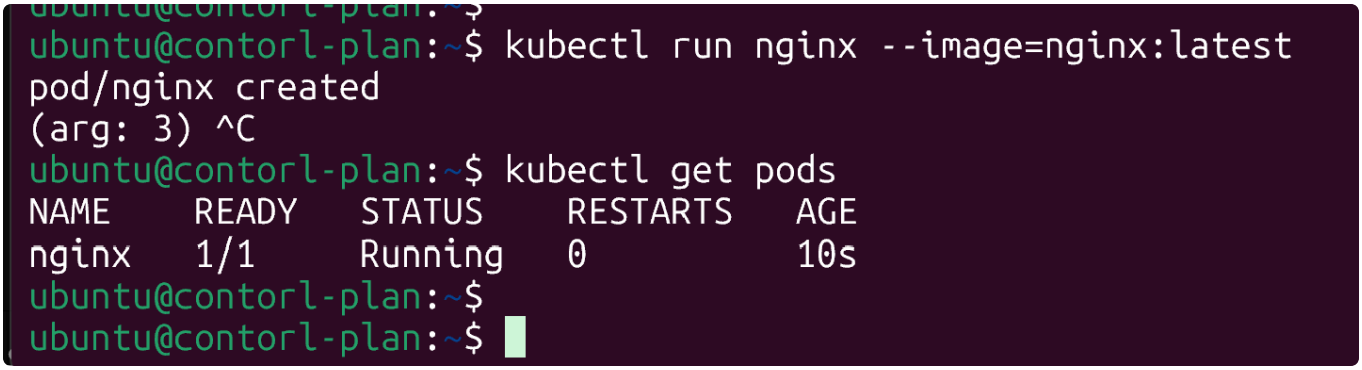


Join worker in the Cluster:

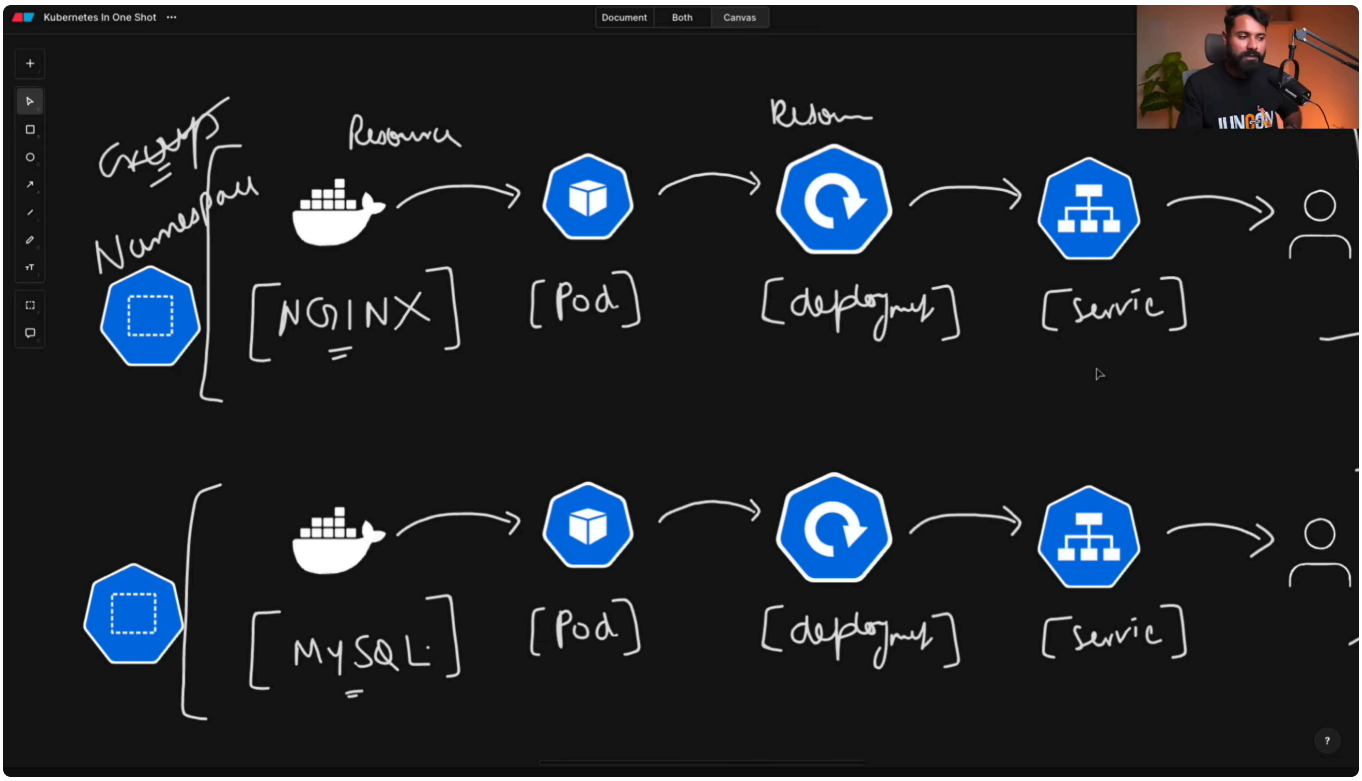




Run a container:



Namespace:

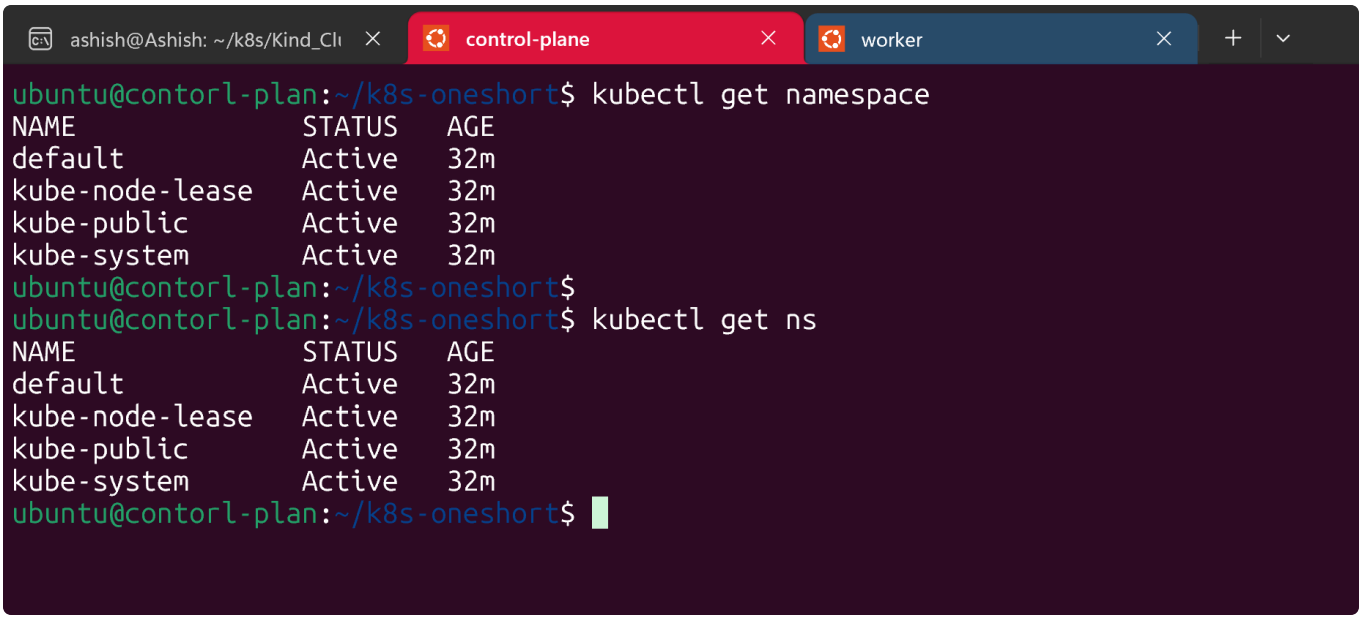


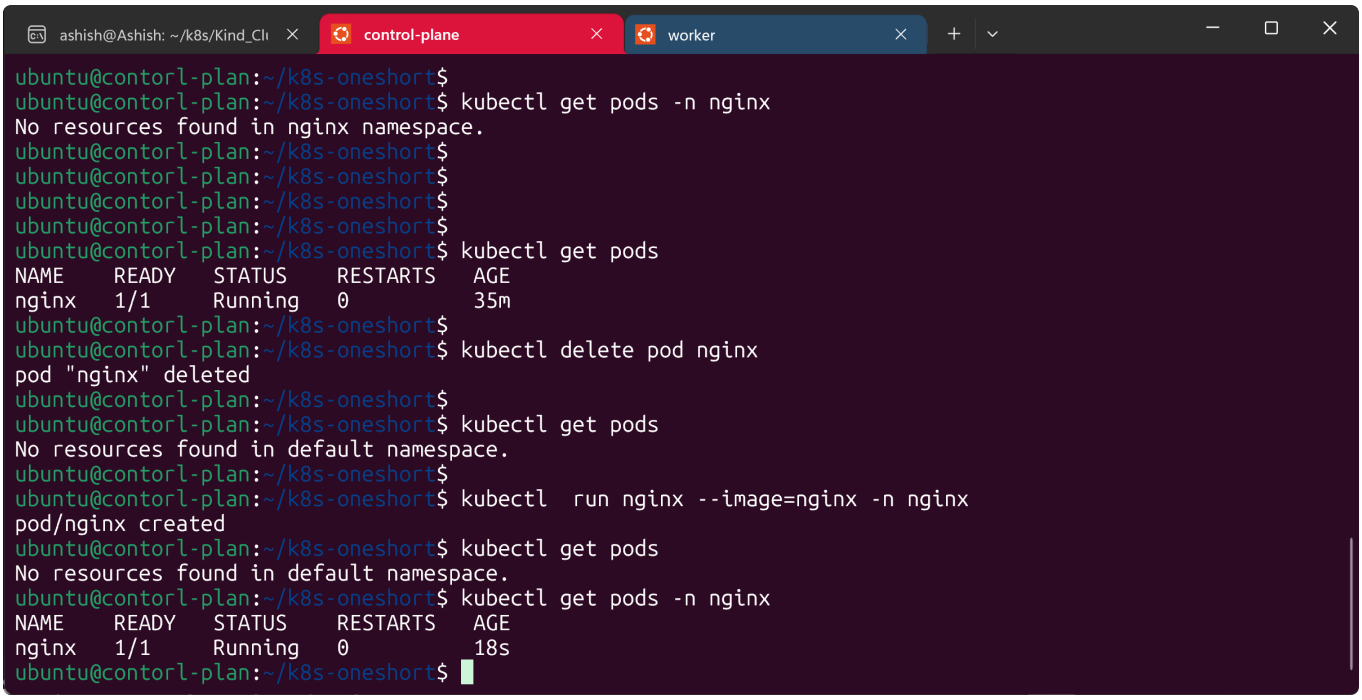

```
kubectl get namespace
#or
kubectl get ns

kubectl create ns <name of namespace>

#for create a nginx pod in default namespace
#command is similar like docker command to run nginx container
kubectl run nginx --image=nginx

kubectl delete pod nginx  #for delete
```





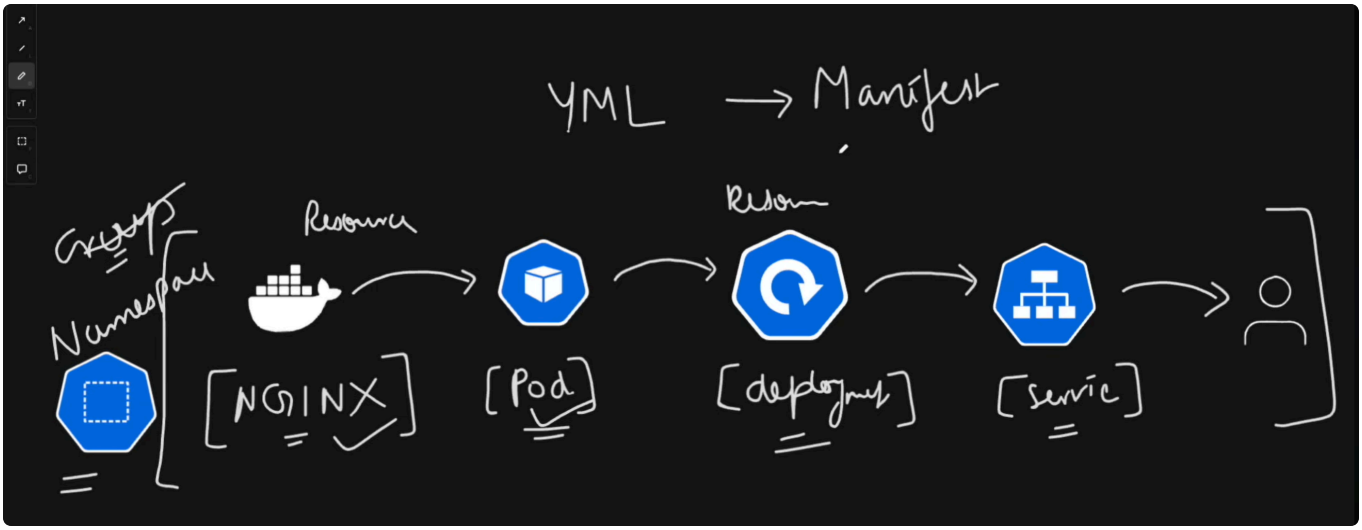

```
#create
kubectl create ns nginx
kubectl run nginx --image=nginx -n nginx #for run in nginx namespace
kubectl get pods -n nginx

#DELETE
kubectl delete pods nginx  -n nginx
kubectl delete ns nginx
```

```
ubuntu@contorl-plan:~$
ubuntu@contorl-plan:~$ kubectl delete pods nginx
Error from server (NotFound): pods "nginx" not found
ubuntu@contorl-plan:~$ kubectl delete pods nginx  -n nginx
pod "nginx" deleted
ubuntu@contorl-plan:~$
ubuntu@contorl-plan:~$ kubectl delete ns nginx
namespace "nginx" deleted
ubuntu@contorl-plan:~$
ubuntu@contorl-plan:~$
ubuntu@contorl-plan:~$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   4h21m
kube-node-lease   Active   4h21m
kube-public       Active   4h21m
kube-system       Active   4h21m
ubuntu@contorl-plan:~$
```

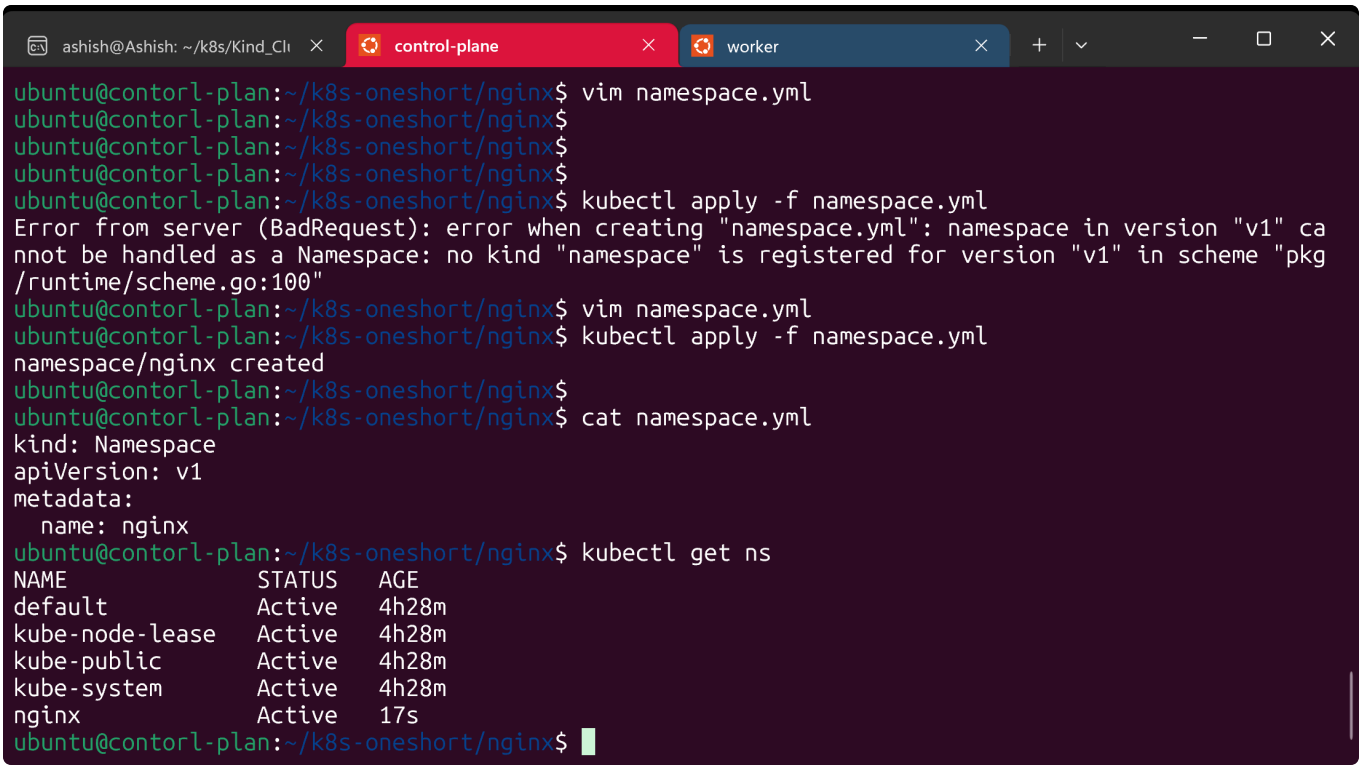
Creating YAML

YAML -> Manifest file



namespace.yml

```yaml
kind: Namespace
apiVersion: v1
metadata:
  name: nginx
```



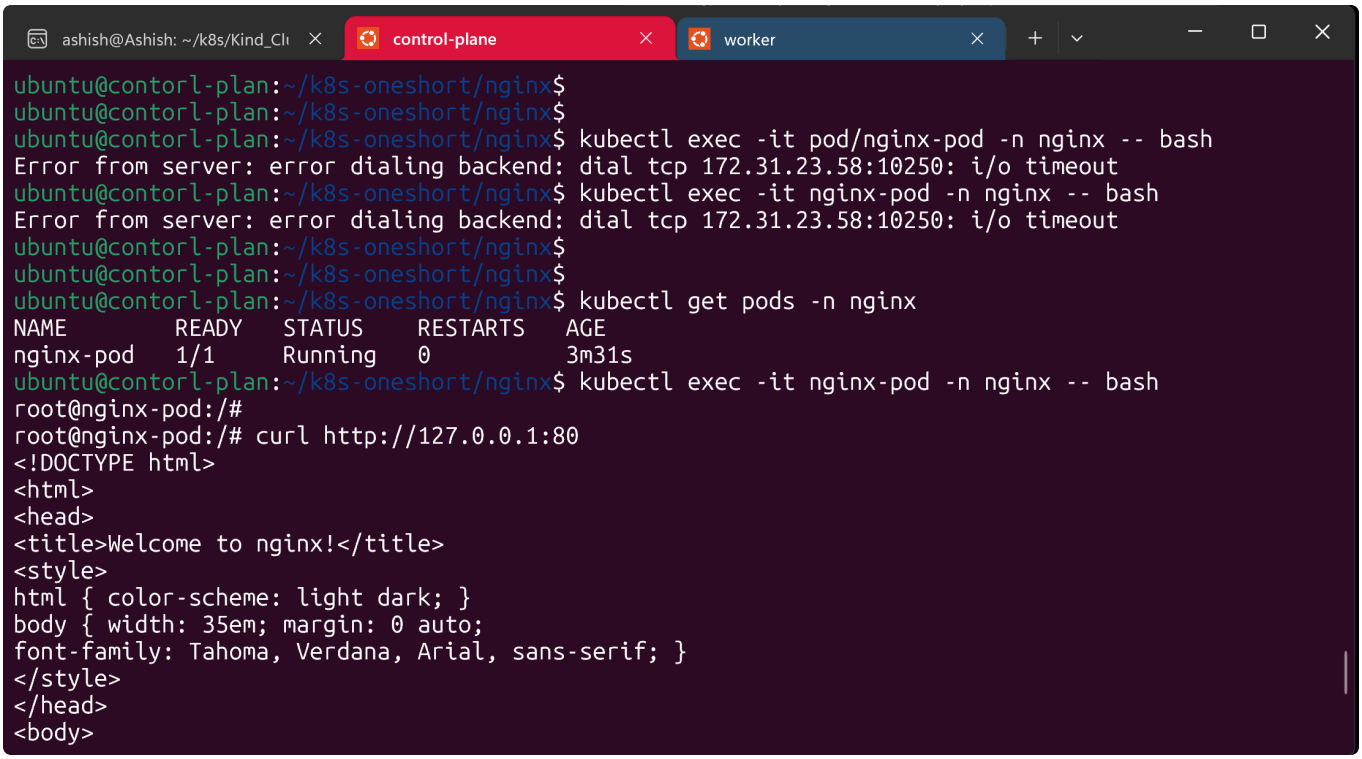
pod.yml:

```yaml
kind: Pod
apiVersion: v1
metadata:
  name: nginx-pod
  namespace: nginx

spec:
  containers:
  -  name: nginx
     image: nginx:latest
     ports:
     - containerPort: 80
```

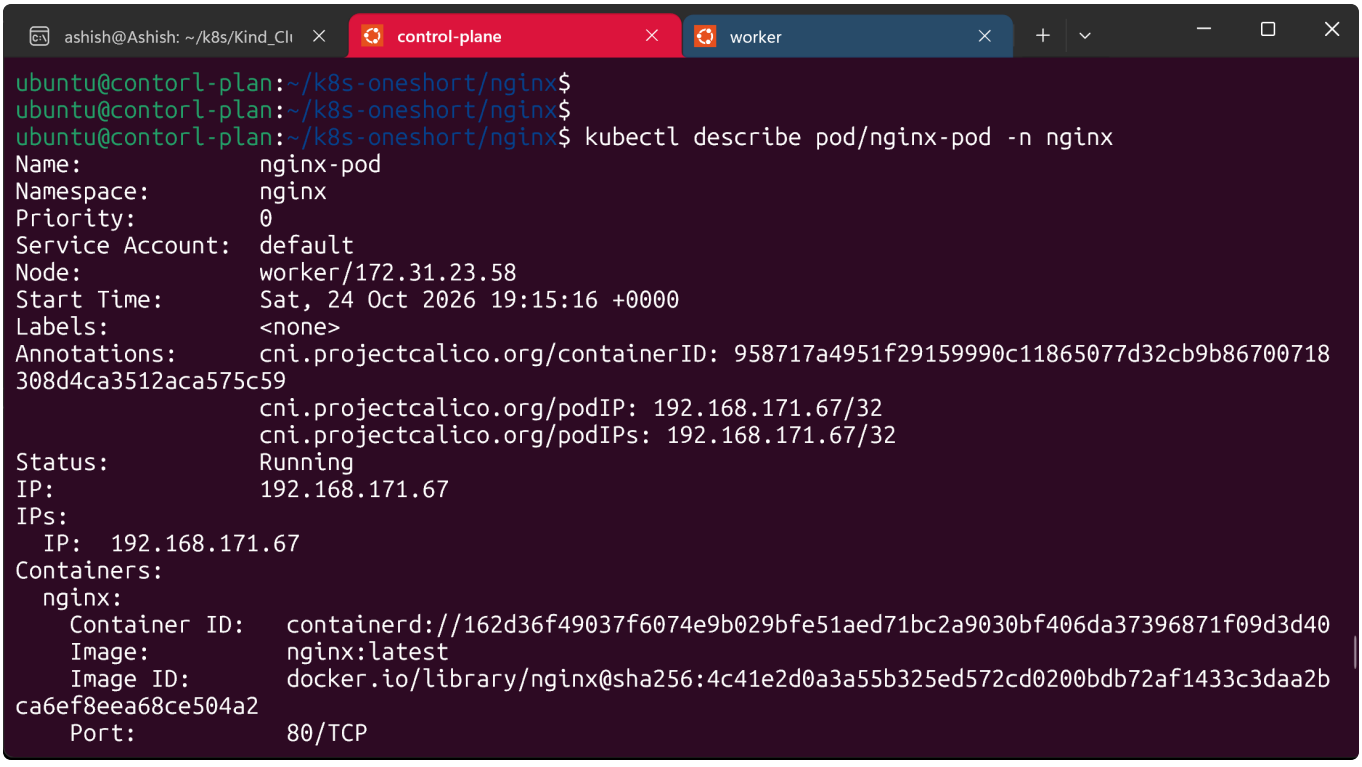
```
kubectl apply -f pod.yml
kubectl get pods -n nginx
```

> *if using AWS add port 10250 in inbound rule of Security Group



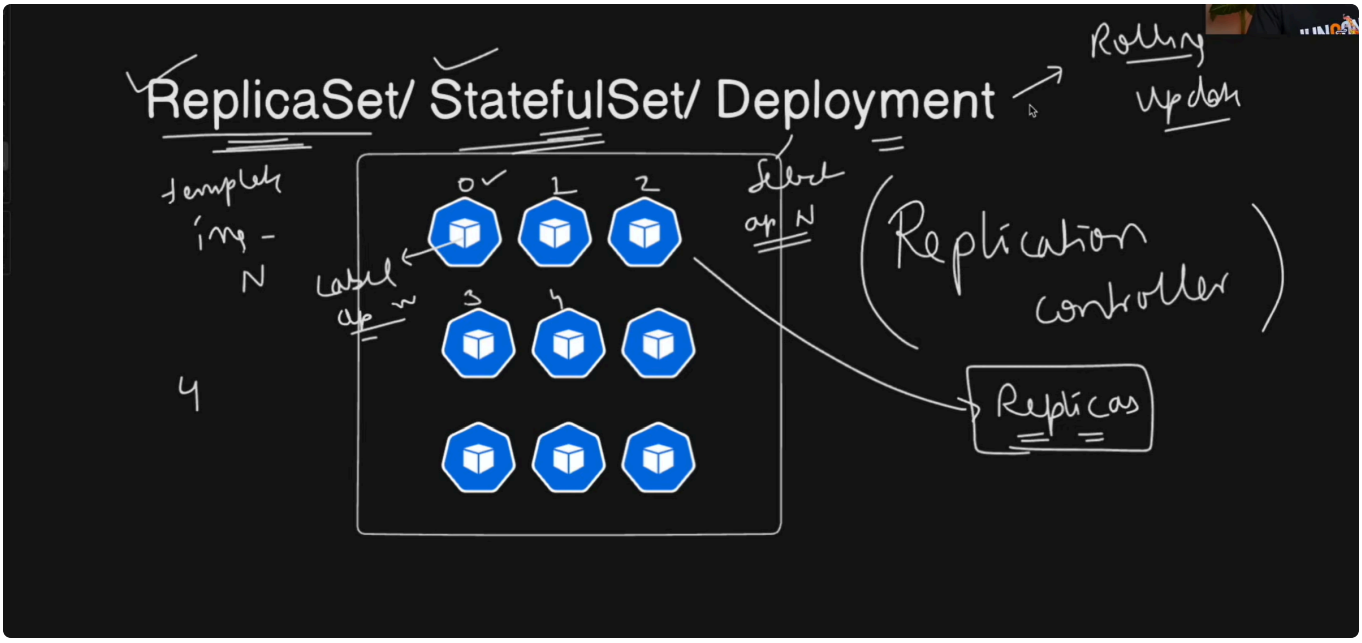

```
kubectl exec -it nginx-pod -n nginx -- bash  #for get into the container
#OR
kubectl exec -it pod/nginx-pod -n nginx -- bash

kubectl describe pod/nginx-pod -n nginx  # similar to docker inspect + logs
```



Workloads:

Deployments, StatefulSets, DaemonSets, ReplicaSets, Jobs, CronJobs.



All these help you to replicate the containers/pods but deployment also help in RollingUpdate (rollout)

> All these are almost similar yml with there own specific advantage:
>
> 1. ReplicaSet
> 2. StatefulSet
> 3. Deployment rollout: while the pod update the some pods are in update state and some are in running not all pods are update in same time this is rollingUpdate

deployment.yml:

```yaml
kind: Deployment
apiVersion: apps/v1
metadata:
  name: nginx-deployment
  namespace:  nginx

spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx

  template:
```

```yaml
    metadata:
      name: nginx-dep-pod
      labels:
        app: nginx

    spec:
      containers:
      - name:  nginx
        image: nginx:latest
        ports:
        -  containerPort: 80
```

```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl apply -f deployment.yml
error: resource mapping not found for name: "nginx-deployment" namespace: "nginx" from "deployment.yml": no matches for kind "Deployment" in v
ersion "app/v1"
ensure CRDs are installed first
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ vim deployment.yml
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl apply -f deployment.yml
Error from server (BadRequest): error when creating "deployment.yml": Deployment in version "v1" cannot be handled as a Deployment: json: cann
ot unmarshal object into Go struct field Container.spec.template.spec.containers.ports of type []v1.ContainerPort
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ vim deployment.yml
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl apply -f deployment.yml
deployment.apps/nginx-deployment created
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6dbd7f7b8c-dqlzn   1/1     Running   0          17s
nginx-deployment-6dbd7f7b8c-ss45m   1/1     Running   0          17s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get deployments.apps
No resources found in default namespace.
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get deployments.apps  -n nginx
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   2/2     2            2           44s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get deployments  -n nginx
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   2/2     2            2           101s
```

how to scale using deployment:

```
kubectl scale deployment/nginx-deployment -n nginx --replicas=5
kubectl get pods -n nginx

kubectl get pods -n nginx o wide  #for more details about pods
```

```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl scale deployment/nginx-deployment -n nginx --re
plicas=5
deployment.apps/nginx-deployment scaled
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6dbd7f7b8c-2gsf2   1/1     Running   0          12s
nginx-deployment-6dbd7f7b8c-5s8cj   1/1     Running   0          12s
nginx-deployment-6dbd7f7b8c-dqlzn   1/1     Running   0          8m38s
nginx-deployment-6dbd7f7b8c-slqwv   1/1     Running   0          12s
nginx-deployment-6dbd7f7b8c-ss45m   1/1     Running   0          8m38s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
```

RollingUpdate:

```
kubectl set image deployment/nginx-deployment -n nginx nginx=nginx:1.27.1
```

```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl set image deployment/nginx-deployment -n nginx nginx=nginx:1.27.3
deployment.apps/nginx-deployment image updated
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-77b9979d7f-6675p   1/1     Running   0          12s
nginx-deployment-77b9979d7f-fzh5m   1/1     Running   0          12s
nginx-deployment-77b9979d7f-pmp2z   1/1     Running   0          12s
nginx-deployment-77b9979d7f-rjfdf   1/1     Running   0          7s
nginx-deployment-77b9979d7f-xkptd   1/1     Running   0          7s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl set image deployment/nginx-deployment -n nginx nginx=nginx:1.27.1
deployment.apps/nginx-deployment image updated
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS              RESTARTS   AGE
nginx-deployment-774f469f5c-5rcqb   0/1     ContainerCreating   0          2s
nginx-deployment-774f469f5c-hj6rs   0/1     ContainerCreating   0          2s
nginx-deployment-774f469f5c-jpd64   0/1     ContainerCreating   0          2s
nginx-deployment-77b9979d7f-6675p   1/1     Running             0          22s
nginx-deployment-77b9979d7f-fzh5m   1/1     Running             0          22s
nginx-deployment-77b9979d7f-rjfdf   1/1     Running             0          17s
nginx-deployment-77b9979d7f-xkptd   1/1     Running             0          17s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-774f469f5c-5rcqb   1/1     Running   0          8s
nginx-deployment-774f469f5c-88t7l   1/1     Running   0          4s
nginx-deployment-774f469f5c-hj6rs   1/1     Running   0          8s
nginx-deployment-774f469f5c-jpd64   1/1     Running   0          8s
nginx-deployment-774f469f5c-srwch   1/1     Running   0          4s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
```

```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl set image deployment/nginx-deployment -n nginx nginx=nginx:latest
deployment.apps/nginx-deployment image updated
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS              RESTARTS   AGE
nginx-deployment-6dbd7f7b8c-gtc69   0/1     ContainerCreating   0          1s
nginx-deployment-6dbd7f7b8c-sn8zg   0/1     ContainerCreating   0          1s
nginx-deployment-6dbd7f7b8c-zgsr5   0/1     ContainerCreating   0          1s
nginx-deployment-774f469f5c-5rcqb   1/1     Running             0          59s
nginx-deployment-774f469f5c-88t7l   1/1     Running             0          55s
nginx-deployment-774f469f5c-jpd64   1/1     Running             0          59s
nginx-deployment-774f469f5c-srwch   1/1     Running             0          55s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS              RESTARTS   AGE
nginx-deployment-6dbd7f7b8c-gtc69   1/1     Running             0          3s
nginx-deployment-6dbd7f7b8c-p2hzf   0/1     ContainerCreating   0          1s
nginx-deployment-6dbd7f7b8c-sn8zg   1/1     Running             0          3s
nginx-deployment-6dbd7f7b8c-xnz6d   0/1     ContainerCreating   0          1s
nginx-deployment-6dbd7f7b8c-zgsr5   1/1     Running             0          3s
nginx-deployment-774f469f5c-srwch   1/1     Running             0          57s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS        RESTARTS   AGE
nginx-deployment-6dbd7f7b8c-gtc69   1/1     Running       0          4s
nginx-deployment-6dbd7f7b8c-p2hzf   1/1     Running       0          2s
nginx-deployment-6dbd7f7b8c-sn8zg   1/1     Running       0          4s
nginx-deployment-6dbd7f7b8c-xnz6d   1/1     Running       0          2s
nginx-deployment-6dbd7f7b8c-zgsr5   1/1     Running       0          4s
nginx-deployment-774f469f5c-srwch   0/1     Terminating   0          58s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6dbd7f7b8c-gtc69   1/1     Running   0          6s
nginx-deployment-6dbd7f7b8c-p2hzf   1/1     Running   0          4s
nginx-deployment-6dbd7f7b8c-sn8zg   1/1     Running   0          6s
nginx-deployment-6dbd7f7b8c-xnz6d   1/1     Running   0          4s
nginx-deployment-6dbd7f7b8c-zgsr5   1/1     Running   0          6s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
```

ReplicaSet:

> *ReplicaSet and Deployment are both same and also there yml. but Deployment have rollingUpdate feature

```
kind: ReplicaSet
apiVersion: apps/v1
metadata:
```

```
  name: nginx-replicaset
  namespace:  nginx

spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      name: nginx-rep-pod
      labels:
        app: nginx

    spec:
      containers:
      - name:  nginx
        image: nginx:latest
        ports:
        -  containerPort: 80
```

```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl apply -f replicasets.yml
replicaset.apps/nginx-replicaset created
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                    READY   STATUS    RESTARTS   AGE
nginx-replicaset-d2qpq   1/1     Running   0          13s
nginx-replicaset-sp78s   1/1     Running   0          13s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get replicasets -n nginx
NAME              DESIRED   CURRENT   READY   AGE
nginx-replicaset   2         2         2       49s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
```

DaemonSet:

> it make sure in all available worker node have atleast one pods is running.

```
kind: DaemonSet
apiVersion: apps/v1
metadata:
  name: nginx-daemonset
  namespace:  nginx
```

```yaml
spec:
  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      name: nginx-dmn-pod
      labels:
        app: nginx

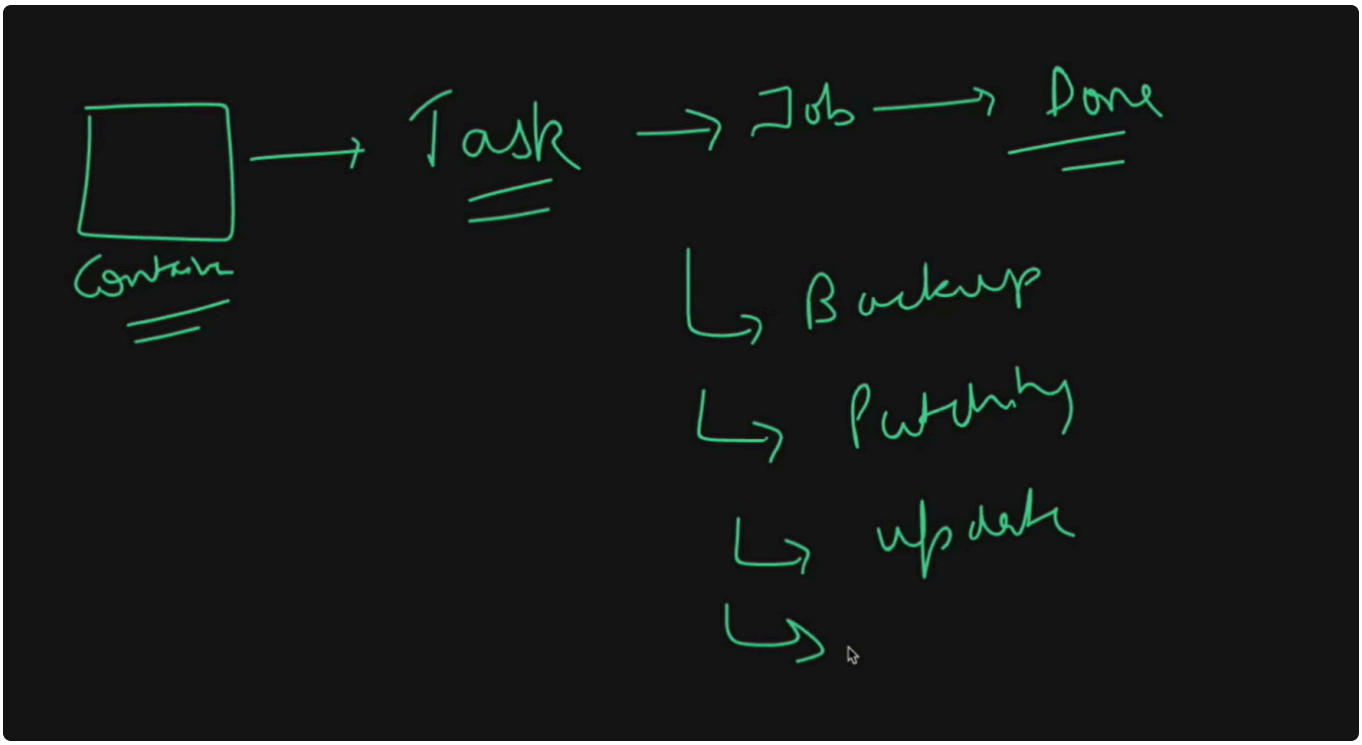
    spec:
      containers:
      - name:  nginx
        image: nginx:latest
        ports:
        -  containerPort: 80
```

> *it automatically create atleast one pod according to the all available worker nodes.

```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl apply -f daemonset.yml
daemonset.apps/nginx-daemonset created
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                    READY   STATUS    RESTARTS   AGE
nginx-daemonset-nq9zl   1/1     Running   0          6s
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
```

Job:

> any task run for a particular period of time. it would be any thing like backup, patching, updating, etc




```
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl apply -f jobs.yml
job.batch/job created
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                    READY   STATUS    RESTARTS   AGE
job-5pjkc               1/1     Running   0          6s
nginx-daemonset-nq9zl   1/1     Running   0          21m
ubuntu@contorl-plan:~/k8s-oneshort/nginx$
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                    READY   STATUS    RESTARTS   AGE
job-5pjkc               1/1     Running   0          11s
nginx-daemonset-nq9zl   1/1     Running   0          21m
ubuntu@contorl-plan:~/k8s-oneshort/nginx$ kubectl get pods -n nginx
NAME                    READY   STATUS      RESTARTS   AGE
job-5pjkc               0/1     Completed   0          13s
nginx-daemonset-nq9zl   1/1     Running     0          21m
```

job.yml:

```yaml
kind: Job
apiVersion: batch/v1
metadata:
  name: job
  namespace: nginx

spec:
  completions: 1
  parallelism: 1
  template:
    metadata:
      name: demo-job-pod
```

```
      labels:
        app: batch-task
    spec:
      containers:
      - name: batch-container
        image: busybox:latest
        command: ["sh", "-c", "echo Hello frinds! && sleep 10"]
      restartPolicy: Never
```

CronJob:

> when a user want to run a job/task for a particular schedule is called a cronjob
> *cron mean following a particular schedule.